

Synchronous FIFO — Design and Verification Report

RTL Design Review & Class-Based SystemVerilog Verification Environment

September 2026

1. Introduction

This report documents the RTL design and SystemVerilog-based functional verification of a **16-deep, 8-bit wide synchronous FIFO** (`syn_FIFO`). It covers the FIFO's theory of operation, the RTL micro-architecture, the class-based testbench built to verify it, the verification methodology and checking strategy, and a review of limitations and possible improvements found while analyzing the code.

2. Background: Synchronous FIFO Theory of Operation

A synchronous FIFO is controlled by a single, free-running clock shared by both the writing and reading logic — unlike an asynchronous FIFO, which uses independent write and read clocks and requires clock-domain-crossing synchronization logic.

Key properties of a synchronous FIFO architecture (as reflected in the supplied reference material on synchronous FIFOs):

- A single free-running clock drives write enable, read enable, and the status flag logic, so `FULL` and `EMPTY` are always synchronized to that one clock — there is no metastability risk between write-side and read-side logic, because there is only one side.
- Writes occur on a defined clock edge when `WRITE ENABLE` (here, `wr`) is asserted; reads occur on a defined clock edge when `READ ENABLE` (here, `rd`) is asserted.
- Because both operations share the same clock domain, a synchronous FIFO is the only FIFO topology where `full` / `empty` status can, in principle, support fully concurrent read/write checking without cross-domain pointer synchronizers (e.g., Gray-code pointer synchronization used in asynchronous FIFOs).

The design analyzed here implements exactly this model: one `clk` domain, one `rst` , and `wr` / `rd` request lines feeding pointer and counter logic that derives `full` and `empty` combinationally from an occupancy count.

3. RTL Design: `syn_FIFO`

3.1 Port List

Port	Direction	Width	Description
<code>clk</code>	input	1	Free-running system clock
<code>rst</code>	input	1	Synchronous, active-high reset
<code>wr</code>	input	1	Write enable / request
<code>rd</code>	input	1	Read enable / request
<code>din</code>	input	8	Write data
<code>dout</code>	output reg	8	Read data (registered)
<code>empty</code>	output	1	FIFO empty status flag

full	output	1	FIFO full status flag
------	--------	---	-----------------------

3.2 Internal Architecture

- **Storage:** `mem[15:0]`, a 16-entry array of 8-bit registers — a circular buffer.
- **Write pointer** `wptr` (4 bits): indexes the next location to write; increments (with natural wraparound, since it is 4 bits wide for 16 locations) on every accepted write.
- **Read pointer** `rptr` (4 bits): indexes the next location to read; increments on every accepted read.
- **Occupancy counter** `cnt` (5 bits, range 0–31 but used 0–16): tracks how many valid entries are currently stored. Incremented on accepted write, decremented on accepted read.
- **Status flags** (combinational, derived from `cnt`):
 - `empty = (cnt == 0)`
 - `full = (cnt == 16)`

3.3 Functional Behavior (per clock edge)

On every `posedge clk`:

1. **If `rst` is asserted** — `wptr`, `rptr`, and `cnt` are all cleared to 0. (Note: `mem` contents and `dout` are *not* explicitly cleared — see §6.1.)
2. **Else if `wr` is asserted and the FIFO is not full** — `din` is written into `mem[wptr]`, `wptr` increments, and `cnt` increments.
3. **Else if `rd` is asserted and the FIFO is not empty** — `mem[rptr]` is registered onto `dout`, `rptr` increments, and `cnt` decrements.

Because this is an `if / else if / else if` chain, **a write request takes priority over a read request if both `wr` and `rd` are asserted in the same cycle**, and if both are asserted, only the write executes that cycle — simultaneous read+write is not supported by this implementation (see §6.2).

3.4 Read Latency

`dout` is a registered output: when `rd` is asserted and accepted on cycle N , the corresponding data appears on `dout` starting at cycle $N+1$ (one-cycle read latency), which the testbench's monitor logic explicitly accounts for (see §4.4).

4. Verification Environment

4.1 Architecture

The testbench follows a class-based, layered verification architecture conceptually similar to UVM, but implemented directly with SystemVerilog `mailbox`s and `events` rather than the UVM class library. The dataflow is:

```

generator --(mailbox: gdmbx)--> driver --(virtual interface pins)--> DUT (syn_FIFO)
                                     |
                                     v
                               monitor (samples pins)
                                     |
                               (mailbox: msmbx)
                                     v
                               scoreboard

```

All four active components (`generator`, `driver`, `monitor`, `scoreboard`) run concurrently, forked from `environment::test()`.

4.2 transaction Class

The stimulus/observation item. Fields:

- `rand bit oper` — randomized operation selector, constrained by `oper_ctrl` to a 50%/50% distribution between `1` (write) and `0` (read), using SystemVerilog's weighted `dist` constraint.
- `rd`, `wr` — captured control bits (used by monitor/scoreboard).
- `data_in` (8 bits), `data_out` (8 bits) — write/read data.
- `full`, `empty` — captured status flags (used by scoreboard to decide whether an operation was legal/expected to succeed).

4.3 generator Class

- Holds a handle to the `gdmbx` mailbox (shared with the driver) and a `count` (set to **90** in the top-level `tb` module) controlling how many transactions are produced.
- `run()` repeats `count` times: randomizes the transaction (`assert (tr.randomize) else $error(...)`), puts it into the mailbox for the driver, logs it, then **blocks on the next event** before generating the next one.
- This `next` event is bound (in `environment::new`) to the same event handle as the scoreboard's `next` event, so the generator only produces a new transaction after the scoreboard has finished checking the previous one — pacing generation to verification throughput.
- Once `count` transactions have been issued, `-> done` is triggered, which `environment::post_test()` waits on to know the test is complete.

4.4 driver Class

- Holds the `virtual fifo_if` handle used to drive DUT pins, and the `gdmbx` mailbox handle to receive transactions from the generator.
- `reset()`: asserts `rst`, deasserts `rd/wr`, clears `data_in`, holds reset for 5 clock cycles, then deasserts `rst`. Run once before the main test loop (`environment::pre_test()`).
- `write()`: on a clock edge, asserts `wr` and drives `data_in` with a random value in `[1, 10]` (`$urandom_range(1, 10)`) — note this data value is generated directly in the driver rather than from the transaction's `data_in` field. Holds `wr` for one cycle, deasserts it, and waits one further idle cycle.
- `read()`: on a clock edge, asserts `rd` for one cycle, deasserts it, and waits one further idle cycle.
- `run()`: forever loop — gets a transaction from `gdmbx`, and dispatches to `write()` if `data.oper == 1` or `read()` otherwise.

4.5 monitor Class

- Holds the `virtual fifo_if` handle and the `msmbx` mailbox handle (shared with the scoreboard).
- `run()`: waits 2 clock edges, then samples `wr`, `rd`, `data_in`, `full`, and `empty` into a fresh `transaction`; waits one more clock edge and *then* samples `data_out` — this extra edge correctly accounts for the DUT's one-cycle registered-read latency described in §3.4.
- Packages the sampled values into a transaction and forwards it to the scoreboard via `msmbx`, printing a `[MON]` trace line.

4.6 scoreboard Class (Checker / Reference Model)

- Maintains `bit [7:0] din[$]` — a SystemVerilog queue acting as a **golden reference model** of FIFO contents.
- On receiving each monitored transaction:
 - **If `tr.wr` is set:**
 - If `tr.full` was `0` (write should have succeeded), the observed `data_in` is pushed onto the front of the reference queue (`din.push_front(...)`).
 - If `tr.full` was `1`, the write is expected to have been ignored by the DUT, and no reference update occurs ("FIFO is full" logged).
 - **If `tr.rd` is set:**

- If `tr.empty` was `0` (read should have succeeded), the oldest entry is popped from the back of the reference queue (`din.pop_back()`) — since new data is pushed to the front and popped from the back, the queue models strict FIFO (first-in-first-out) ordering — and compared against the DUT’s observed `data_out`. A mismatch increments the `err` counter and raises `$error`.
- If `tr.empty` was `1`, no read was expected to succeed (“FIFO IS EMPTY” logged), and no comparison is made.
 - `-> next` is triggered at the end of every iteration, releasing the generator to produce its next transaction (§4.3).
- `err` accumulates the total number of data mismatches detected over the run and is reported at the end of simulation.

4.7 environment Class

- Instantiates and connects all components: creates `gdmbox` and `msmbx` mailboxes, constructs `generator/driver` around `gdmbox` and `monitor/scoreboard` around `msmbx`, assigns the shared virtual interface (`fif`) to `driver` and `monitor`, and binds the shared pacing event `nextgs` to both `gen.next` and `sco.next`.
- `pre_test()` — calls `drv.reset()`.
- `test()` — forks all four component `run()` tasks together with `join_any` (the block completes as soon as **any one** finishes — in practice, this is the generator once it has issued all 90 transactions and triggered `done`; note `driver`, `monitor`, and `scoreboard` all run `forever` loops with no natural termination).
- `post_test()` — waits for `gen.done.triggered`, then prints the final error count and calls `$finish()`.
- `run()` — sequences `pre_test()` → `test()` → `post_test()`.

4.8 Top-Level Testbench Module (`tb`)

- Instantiates the `fifo_if` interface and the `syn_FIFO` DUT, connecting them via positional port mapping: `dut (fif.clock, fif.rst, fif.wr, fif.rd, fif.data_in, fif.data_out, fif.empty, fif.full)`, matching the DUT’s port order (`clk, rst, wr, rd, din, dout, empty, full`).
- Generates a clock with a 20 ns period (`always #10 fif.clock <= ~fif.clock`), starting from an initial value of `0`.
- Constructs the `environment`, sets `env.gen.count = 90` (90 randomized write/read transactions), and calls `env.run()`.
- Sets up waveform dumping to `dump.vcd` via `$dumpfile / $dumpvars` for post-simulation waveform viewing (e.g., in GTKWave).

5. Verification Methodology Summary

Aspect	Approach
Stimulus generation	Constrained-random (<code>dist</code> constraint, 50/50 write vs. read)
Stimulus data values	Driver-generated via <code>\$urandom_range(1, 10)</code> for writes
Communication	SystemVerilog <code>mailbox #(transaction)</code>
Synchronization/pacing	SystemVerilog named <code>event s (next , done)</code>
Checking strategy	Self-checking scoreboard with a queue-based golden reference model
Pass/fail criteria	<code>err</code> count of <code>DATA MISMATCH</code> events over 90 transactions; ideally 0
Coverage collection	None present in the supplied code (no covergroups)
Waveform capture	VCD dump (<code>dump.vcd</code>) via <code>\$dumpvars</code>
Run length	90 transactions (<code>env.gen.count = 90</code>), configurable at the top module

6. Review Findings: Limitations and Suggested Improvements

These observations come directly from analyzing the provided RTL and testbench code; they are not implemented in the supplied sources but are worth considering before using this design/testbench as a verification sign-off vehicle.

6.1 RTL: Incomplete Reset

Only `wptr`, `rpwr`, and `cnt` are cleared on `rst`. The `mem` array and the `dout` register are **not** reset, so `dout` and stale FIFO memory contents are undefined/retain previous values immediately after reset until overwritten by real traffic. This is functionally acceptable for many FIFOs (data validity is normally gated by `empty`), but is worth flagging explicitly for any downstream user of the design.

6.2 RTL: No Simultaneous Read + Write

The `if / else if / else if` structure means that when `wr` and `rd` are both asserted in the same cycle, **only the write executes**; the read request is silently dropped that cycle. A fully concurrent synchronous FIFO (as referenced in §2) would typically support simultaneous read-while-write in the same cycle when neither `full` nor `empty` blocks it. This is a design choice/limitation to be aware of, not a bug in the current spec, but the testbench does not appear to specifically stress this same-cycle case (the driver's `write() / read()` tasks are mutually exclusive per transaction, so `wr` and `rd` are never asserted together by this testbench).

6.3 Testbench: `assert (tr.randomize)` Missing Parentheses

`randomize` is a built-in class method and is conventionally invoked as `tr.randomize()`. Some simulators tolerate `tr.randomize` without parentheses in specific contexts, but this is nonstandard and can be flagged by lint tools or fail to compile in stricter simulators. Using `tr.randomize()` explicitly is recommended.

6.4 Testbench: Driver Ignores `transaction.data_in`

The `write()` task generates its own random data via `$urandom_range(1, 10)` rather than using the `data_in` field already present on the `transaction` class. This works correctly (the scoreboard reads the *actual* driven `data_in` value back from the DUT interface via the monitor, not from the transaction object), but it means the `transaction` class's `data_in` field is effectively unused as a stimulus vector — an opportunity to make write data part of the randomized, constrainable transaction instead of being hardcoded in the driver.

6.5 No Functional Coverage

The environment has no covergroups/coverpoints, so there's no measurement of how well the 90 random transactions exercised corner cases (e.g., back-to-back writes until `full`, back-to-back reads until `empty`, read-on-empty, write-on-full). Adding a coverage model would strengthen confidence that the constrained-random stimulus is meaningfully exercising the DUT's boundary conditions.

6.6 `join_any` Termination Semantics

`environment::test()` uses `fork ... join_any`, which unblocks as soon as the first forked process completes. In practice this is the `generator` (the only process with a natural endpoint, via its `repeat (count)` loop); `driver`, `monitor`, and `scoreboard` all run `forever` loops. This works because `post_test()` separately waits on `gen.done.triggered` before calling `$finish()`, but the `driver / monitor / scoreboard` processes are effectively left running (and are only terminated by the final `$finish()` call ending the simulation entirely) rather than being gracefully shut down.

6.7 Reset Coverage in Test

`pre_test()` only resets the DUT once at the very start of the run. There is no mid-test reset stimulus, so reset-recovery behavior mid-stream (e.g., resetting a partially-filled FIFO) is not exercised by this testbench.

7. Conclusion

The design implements a standard, easily-understood synchronous FIFO using a circular buffer, dual pointers, and an occupancy counter, matching the textbook single-clock-domain synchronous FIFO architecture referenced in the supporting documentation. The accompanying testbench is a well-structured, layered, class-based verification environment (mirroring UVM's generator/driver/monitor/scoreboard pattern using native SystemVerilog constructs) that self-checks FIFO ordering via a queue-based golden reference model. The implementation is functionally sound for its stated scope, with the limitations noted in §6 representing natural next steps (reset completeness, concurrent read/write support, functional coverage, and driver/transaction data-flow cleanup) for anyone extending this into a more rigorous or production-grade verification IP.